आई आई टी हैदराबाद
IIT Hyderabad



Introduction to Databases

Database System Concepts
Silberschatz, Korth and Sudarshan

What is a DBMS?

- DBMS = Database Management System
- Database: A large integrated collection of data.
- DBMS contains information about a particular enterprise
  - Collection of interrelated data
  - Set of programs to access the data
  - An environment that is both *convenient* and *efficient* to use

Who Uses a DBMS?

- In short: everyone
  - Banking: transactions
  - Airlines: reservations, schedules
  - Universities: registration, grades
  - Sales: customers, products, purchases
  - Online retailers: order tracking, customized recommendations
  - Manufacturing: production, inventory, orders, supply chain
  - Human resources: employee records, salaries, tax deductions

- How many databases have you used so far today?

University Database Example

- Application program examples
  - Add new students, instructors, and courses
  - Register students for courses, and generate class rosters
  - Assign grades to students, compute grade point averages (GPA) and generate transcripts

- In the early days, database applications were built directly on top of file systems

Drawbacks of Using File Systems to Store Data

- Data redundancy and inconsistency
  - Multiple file formats, duplication of information in different files
- Difficulty in accessing data
  - Need to write a new program to carry out each new task
- Data isolation
  - Multiple files and formats
- Integrity problems
  - Integrity constraints (e.g., account balance > 0) become “buried” in program code rather than being stated explicitly
  - Hard to add new constraints or change existing ones



Drawbacks of Using File Systems to Store Data (Cont.)

- Atomicity of updates
  - Failures may leave database in an inconsistent state with partial updates carried out
  - Example: Transfer of funds from one account to another should either complete or not happen at all

- Concurrent access by multiple users



  - Concurrent access needed for performance
  - Uncontrolled concurrent accesses can lead to inconsistencies

- Security problems
  - Hard to provide user access to some, but not all, data

Database systems offer solutions to all the above problems

Why Use a DBMS?

- Data independence
- Efficient access
- Reduced application development time
- Uniform data administration
- Data integrity and security
- Concurrent access
- Recovery from crashes



Why Study Databases?



- Data is useless without the tools to extract information from the data (queries)
  - "Optimal" pricing of an airline ticket
- Datasets are increasing in diversity and volume.
  - Websites, digital libraries, interactive video, Human Genome project, mobile applications
  - Need for DBMS is exploding
- Databases touch most of CS
  - OS, languages, theory, AI, multimedia, logic, …

Levels of Abstraction

- **Physical Level:** Describes how a record (e.g., student) is stored.
- **Logical Level:** Describes data stored in database, and the data relationships.

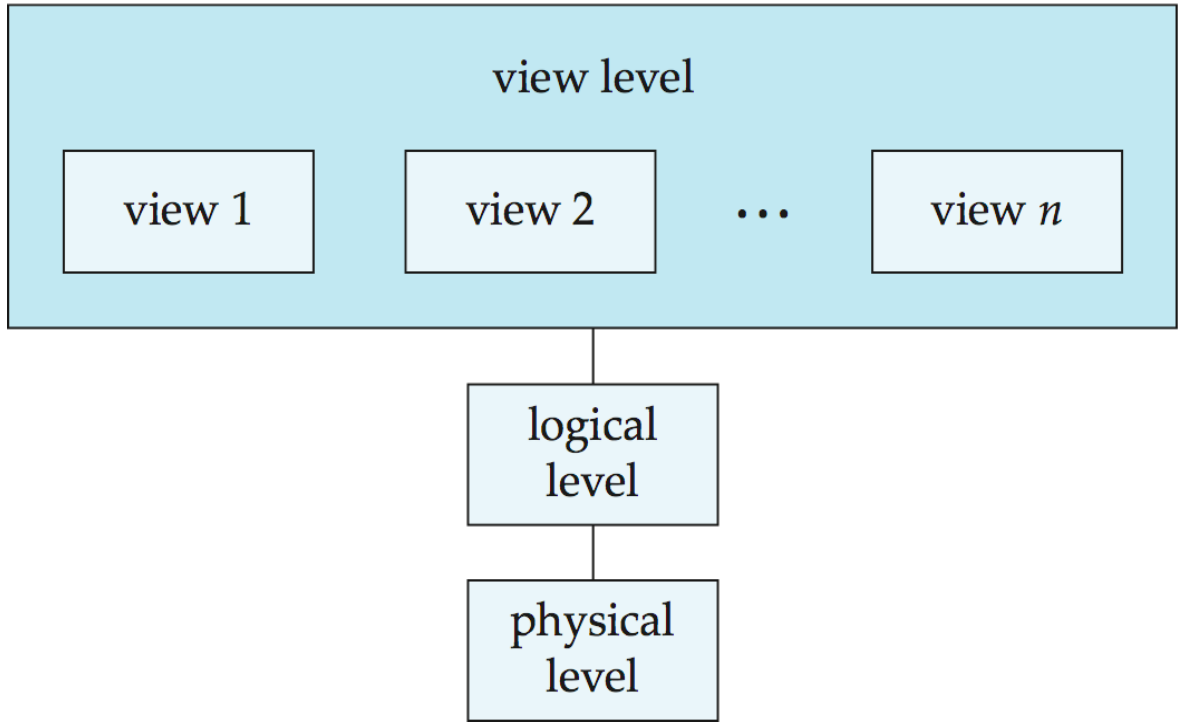
```
type instructor = record
        ID: string;
        name: string;
        dept_name: string;
        salary: integer;
    end;
```



- **View Level:** Application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.

View of Data

- Physical schema describes the files and indexes used.
- Logical schema defines logical structure
- External schema (views) describe how users see the data
- Many external schemas,
  1 conceptual (logical) schema &
  1 physical schema.



An architecture for a database system

Instances and Schemas

- Similar to types and variables in programming languages

- **Schema**: The logical structure of the database
  - Example: The database consists of information about a set of students and instructors and the relationship between them
  - Analogous to type information of a variable in a program
  - **Physical schema**: Database design at the physical level
  - **Logical schema**: Database design at the logical level

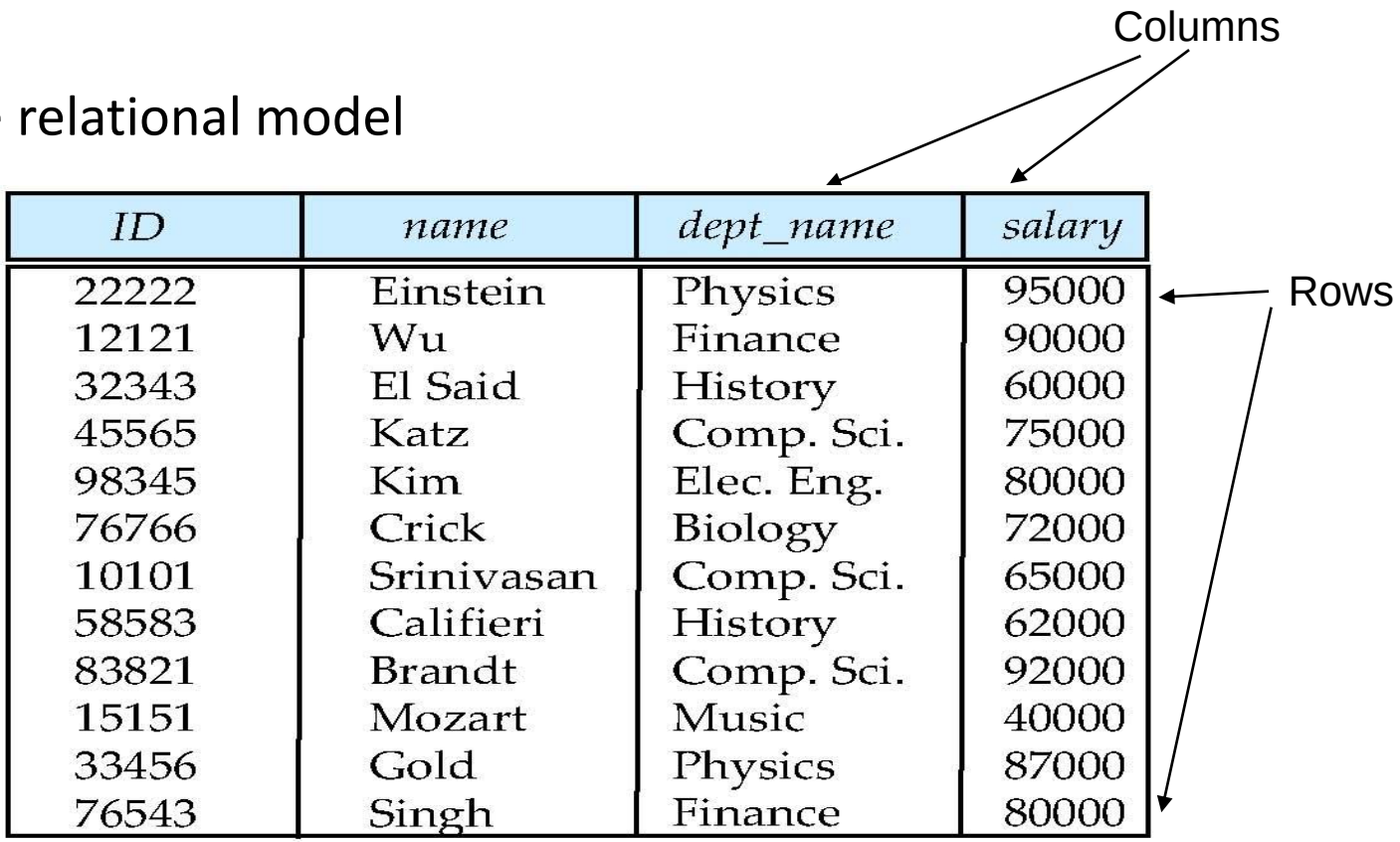
- **Instance**: The actual content of the database at a particular point in time
  - Analogous to the value of a variable

Data Models

- A collection of tools for describing
  - Data
  - Data relationships
  - Data semantics
  - Data constraints
- Entity-Relationship data model (mainly for database design)
- Different data models
  - **Relational model**
  - Object-based data models (Object-oriented and Object-relational)
  - Semi-structured data model (XML)
  - Network model
  - Hierarchical model

Relational Model

- Relational model (Chapter 2)
- Example of tabular data in the relational model



ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

A Sample Relational Database

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Data Manipulation Language (DML)

- Language for accessing and manipulating the data organized by the appropriate data model
  - DML also known as query language

- Two classes of languages
  - **Procedural** – user specifies what data is required and how to get those data
  - **Declarative (non-procedural)** – user specifies what data is required without specifying how to get those data

- SQL is the most widely used query language

Data Definition Language (DDL)

- Specification notation for defining the database schema

  Example: **create table** *instructor* (
  ID **char**(5),
  name **varchar**(20),
  dept_name **varchar**(20),
  salary **numeric**(8,2))

- DDL compiler generates a set of table templates stored in a **data dictionary**
- Data dictionary contains metadata (i.e., data about data)
  - Database schema
  - Integrity constraints: Primary key, referential integrity
  - Authorization

SQL

- **SQL**: A widely used non-procedural language

  Example: Find the ID and building of instructors in the Physics dept.

```
select instructor.ID, department.building
from instructor, department
where instructor.dept_name = department.dept_name and
        department.dept_name = 'Physics'
```

- Application programs generally access databases through one of
  - Language extensions to allow embedded SQL
  - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database

Database Design

The process of designing the general structure of the database:

- **Logical Design** – Deciding on the database schema. Database design requires that we find a "good" collection of relation schemas.
  - Business decision – What attributes should we record in the database?
  - Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?

- **Physical Design** – Deciding on the physical layout of the database

Database Design?

- Is there any problem with this design?

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

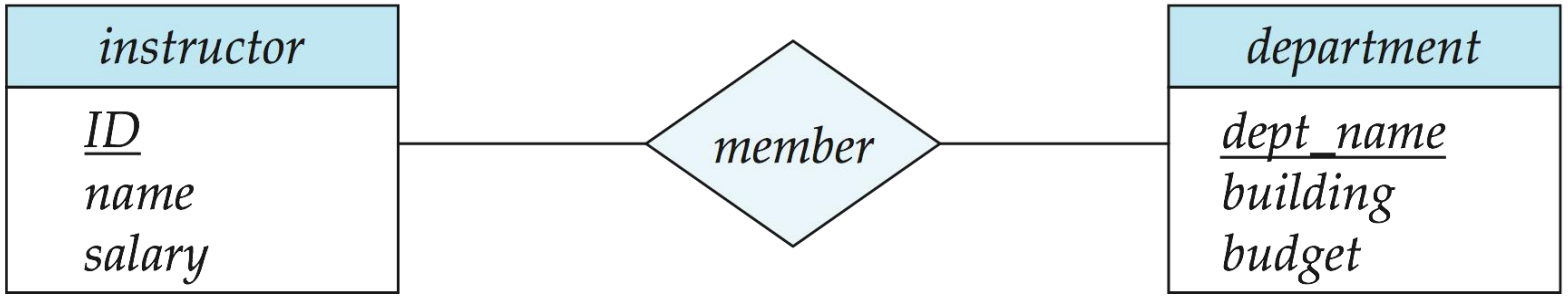
Design Approaches

- Normalization Theory
  - Formalize what designs are bad, and test for them

- Entity Relationship Model
  - Models an enterprise as a collection of *entities* and *relationships*
    - **Entity**: A “thing” or “object” in the enterprise that is distinguishable from other objects
      - Described by a set of *attributes*
    - **Relationship**: An association among several entities
  - Represented diagrammatically by an *entity-relationship diagram*

The Entity-Relationship Model

- Entity Relationship Model
  - Models an enterprise as a collection of *entities* and *relationships*
    - **Entity**: A “thing” or “object” in the enterprise that is distinguishable from other objects
      - Described by a set of *attributes*
    - **Relationship**: An association among several entities
  - Represented diagrammatically by an *entity-relationship diagram*



Storage Management

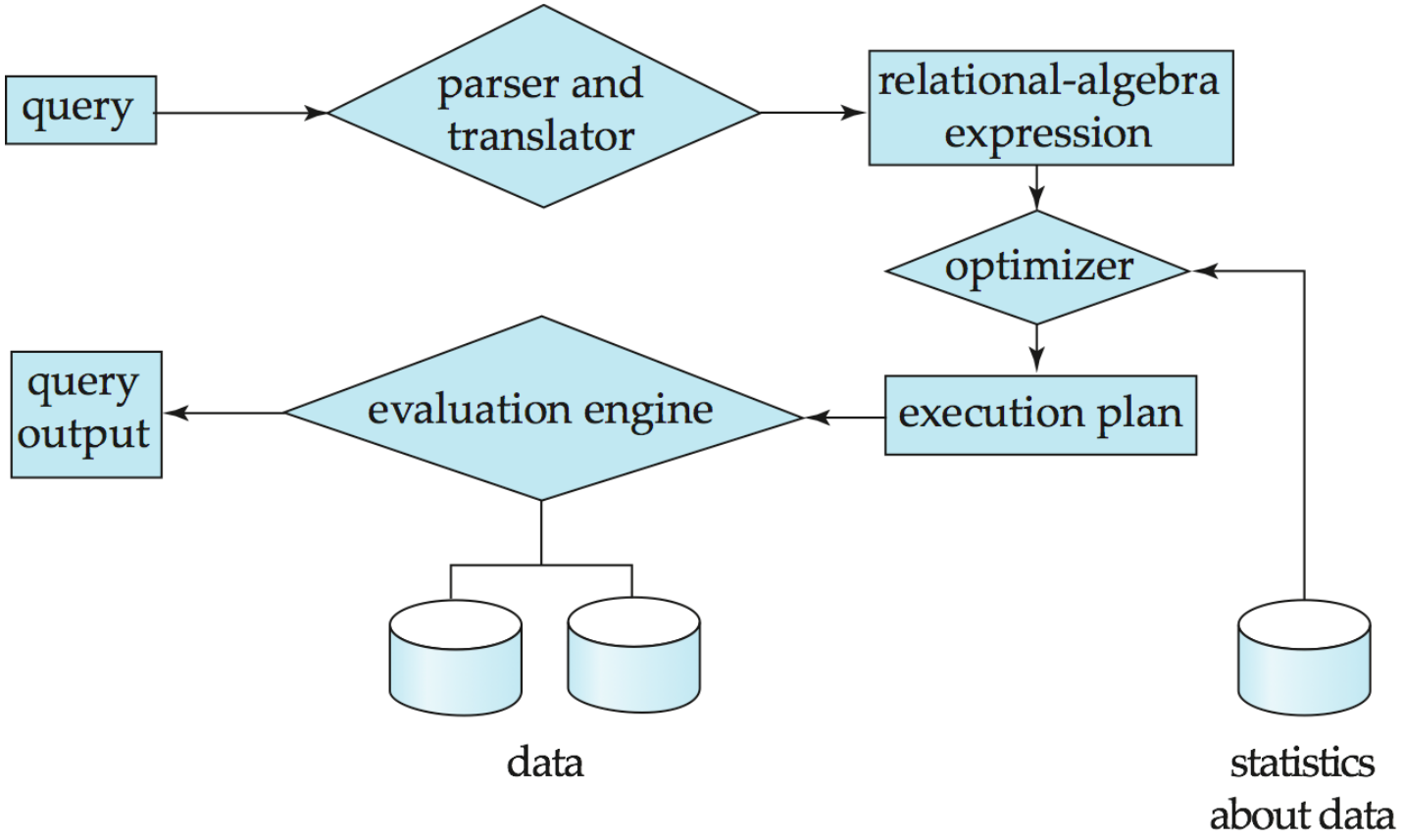
- **Storage manager** is a program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.

- The storage manager is responsible to the following tasks:
  - Interaction with the file manager
  - Efficient storing, retrieving and updating of data

- Issues:
  - Storage access
  - File organization
  - Indexing and hashing

Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation



Query Processing (Cont.)

- Alternative ways of evaluating a given query
  - Equivalent expressions
  - Different algorithms for each operation
- Cost difference between a good and a bad way of evaluating a query can be enormous
- Need to estimate the cost of operations
  - Depends critically on statistical information about relations which the database must maintain
  - Need to estimate statistics for intermediate results to compute cost of complex expressions

Transaction Management

- What if the system fails?
- What if more than one user is concurrently updating the same data?

- A **transaction** is a collection of operations that performs a single logical function in a database application

- **Transaction-management component** ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.

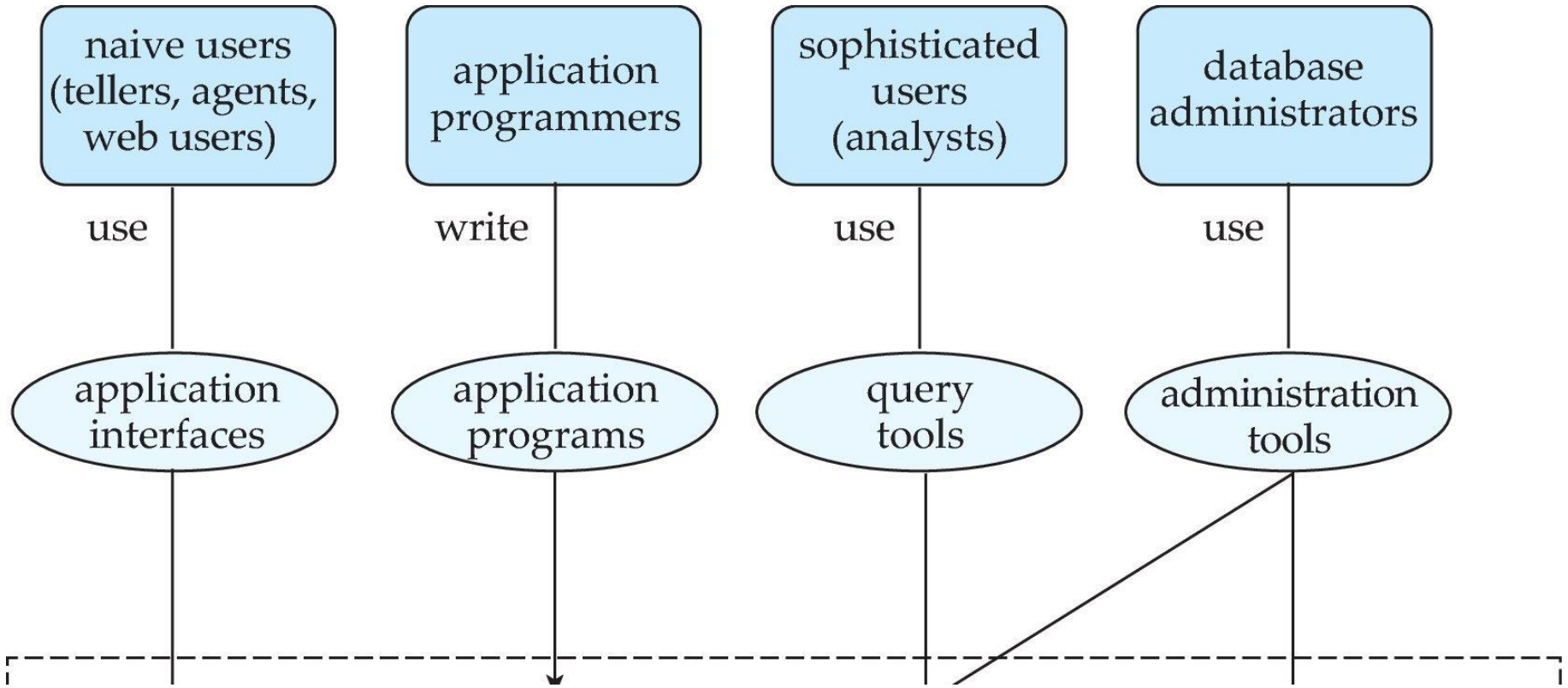
- **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database.

Lots of People use DBMS ...

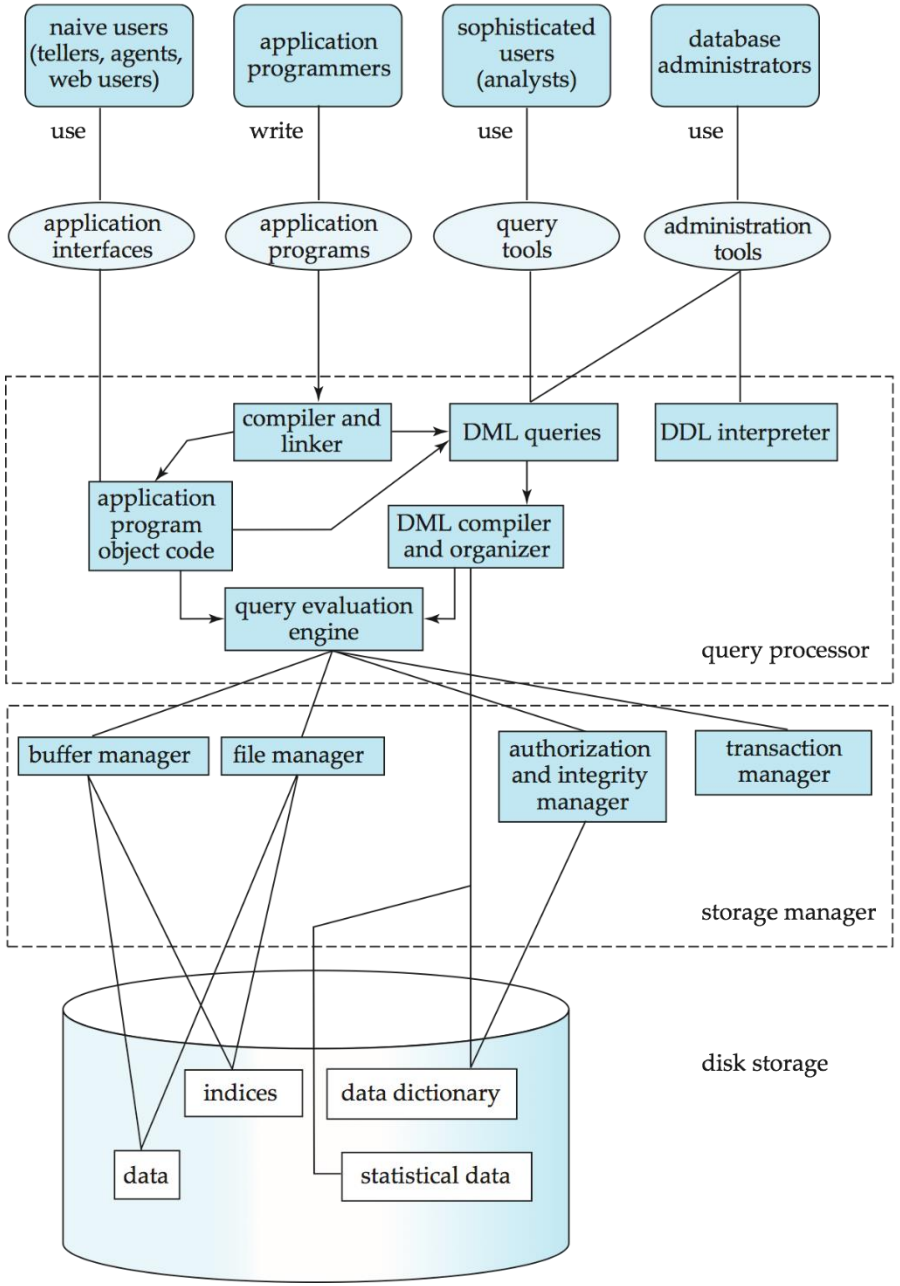
- DBMS vendors
- DB application programmers
  - E.g. smart webmasters
- Database administrator (DBA)
  - Designs logical /physical schemas
  - Handles security and authorization
  - Data availability, crash recovery
  - Database tuning as needs evolve

Must understand how a DBMS works!

Database Users and Administrators



Overall System Architecture



Database Architecture

- The architecture of a database systems is greatly influenced by the underlying computer system on which the database is running:
  - Centralized
  - Client-server
  - Parallel (multi-processor)
  - Distributed

History of Database Systems

- 1950s and early 1960s:
  - Data processing using magnetic tapes for storage
    - Tapes provided only sequential access
  - Punched cards for input

- Late 1960s and 1970s:
  - Hard disks allowed direct access to data
  - Network and hierarchical data models in widespread use
  - Ted Codd defines the relational data model
    - Would win the ACM Turing Award for this work
    - IBM Research begins System R prototype
    - UC Berkeley begins Ingres prototype
  - High-performance (for the era) transaction processing

History of Database Systems (cont.)

- 1980s:
  - Research relational prototypes evolve into commercial systems
    - SQL becomes industrial standard
  - Parallel and distributed database systems
  - Object-oriented database systems

- 1990s:
  - Large decision support and data-mining applications
  - Large multi-terabyte data warehouses
  - Emergence of Web commerce

- Early 2000s:
  - XML and XQuery standards
  - Automated database administration

- Later 2000s:
  - Giant data storage systems
    - Google BigTable, Yahoo PNUTS, Amazon, ..

CYU

- Which of these are more suitable for storing in a DBMS rather than files in an OS? Select all that apply.
  a) Historical stock market prices
  b) Grades for students at the university
  c) Source code for a program
  d) Contents of a textbook

CYU

- When is relational model appropriate for representing data?
  a) When the data can be expressed in the form of tables
  b) For text files
  c) For representing object-oriented models with inheritance, etc.

Summary

- DBMS is used to maintain, query large datasets
- Benefits include recovery from system crashes, concurrent access, quick application development, data integrity and security
- Levels of abstraction give data independence
- DBAs hold responsible, interesting, well-paid jobs
- DBMS R&D is one of the most exciting areas in CS